

Article

A Lightweight Multi-Classification Intrusion Detection Model for Edge IoT Networks

Wei Gao ^{1,2}, Mingyue Wang ^{1,2}, Yadong Pei ^{1,2}, Fangwei Li ³ and Chaonan Wang ^{1,2,*}

¹ Key Laboratory of Public Big Data Security Technology, Chongqing College of Mobile Communication, Chongqing 401420, China; gaowei@cqytxy.edu.cn (W.G.)

² Chongqing Key Laboratory of Public Big Data Security Technology, Chongqing 401420, China

³ School of Communication and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China

* Correspondence: wangchaonan@cqytxy.edu.cn

Abstract

Intrusion detection aims to effectively detect abnormal attacks in Internet of Things (IoT) networks, which is crucial for cybersecurity. However, it is difficult for traditional intrusion detection methods to effectively extract data features from traffic data, and most existing models are too complex to be deployed on edge servers. Addressing this need, this paper proposes a hybrid feature selection method and a lightweight deep learning intrusion detection model. Firstly, the data feature space is reduced using variance filtering, mutual information, and the Pearson Correlation Coefficient, thereby reducing the computational cost of subsequent model training. Then, an intrusion detection model based on a Temporal Convolutional Network (TCN) is constructed. This model utilizes dilated causal convolutions to effectively capture long-term temporal dependencies in network traffic. Simultaneously, the residual connections are used to mitigate the vanishing gradient problem, making the model easier to train and converge. Finally, experiments are conducted on the newly released Edge-IIoTset dataset. The results show that the proposed feature selection algorithm maintains good detection performance despite a significant reduction in feature dimensionality. Furthermore, compared with other models, the proposed TCN-based approach achieves higher classification accuracy with lower computational overhead, demonstrating its suitability for deployment in resource-constrained edge computing environments.

Keywords: IoT security; intrusion detection; feature reduction; temporal convolution networks; mutual information; Pearson Correlation Coefficient



Academic Editor: Marina Prvan

Received: 27 January 2026

Revised: 12 February 2026

Accepted: 16 February 2026

Published: 25 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

With the continuous advancement of information technology, the Internet of Things (IoT) has gradually integrated into all aspects of society and has shown extensive application value in many fields, such as industrial IoT [1], healthcare [2], smart cities [3], and smart homes [4]. However, due to the characteristics of IoT devices, including simplified hardware design, limited computing and storage resources, diverse network protocols, and decentralized geographical deployment, IoT devices are extremely vulnerable to network attacks in practical applications, causing serious network security incidents such as network system paralysis and user privacy leaks [5,6]. Therefore, how to effectively ensure the security and stable operation of IoT systems has become an important issue that urgently needs to be addressed.

To address the threat of network attacks, network intrusion detection systems (NIDS) have emerged as an important part of the proactive defense system. NIDS can identify potential attack behaviors and abnormal access by real-time monitoring and analysis of system logs, user behavior, and network traffic. Research on NIDS began globally in the 1980s and has now become an indispensable part of network security [7]. However, with the continuous development of digital technology and the increasing diversification of network attack methods, the data characteristics of network traffic have become more and more complex, and traditional NIDS can no longer meet the needs of IoT users to accurately identify anomalies and unknown attacks [8].

With the development of artificial intelligence, the NIDS field has begun to use deep learning to extract traffic features, thereby automatically identifying abnormal traffic in complex data traffic. Commonly used deep learning algorithms mainly include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Long Short-Term Memory Networks (LSTMs) [9]. However, these methods generally suffer from complex network structures and large parameter scales, thus requiring high hardware resources for the deployment environment. Meanwhile, traditional NIDS are mostly deployed in the cloud or on central servers. Although they possess strong computing and storage capabilities, they suffer from high latency and high bandwidth consumption in IoT scenarios. To address this, researchers have proposed deploying intrusion detection systems at the edge, enabling traffic analysis and detection to be completed locally. This effectively reduces communication latency, alleviates network transmission pressure, and improves the response speed to local attacks [10].

The introduction of edge-based intrusion detection provides a new assurance for the real-time performance and security of IoT networks. However, it also brings several challenges that urgently need to be addressed:

1. IoT network traffic data usually exhibits high dimensionality and strong feature correlation. If such data are directly fed into the detection model, redundant information will significantly increase computational burden and make the model prone to overfitting, which in turn degrades detection accuracy.
2. Most deep learning-based intrusion detection models require substantial computational and storage resources. Given the resource-constrained nature of edge devices, models with large parameter sizes and high computational costs are difficult to deploy and run efficiently in practical edge environments.

To address the aforementioned issues, this paper aims to research a lightweight intrusion detection method for IoT networks at the edge. The main contributions are as follows:

1. A hybrid feature extraction strategy: Aiming at the issues of high data dimensionality and feature redundancy in IoT network traffic, a hybrid feature selection method integrating variance filtering, mutual information, and correlation analysis is proposed. Firstly, variance filtering is used to quickly remove the information features with small variation amplitude and limited discrimination ability, thereby reducing the computational overhead of subsequent feature selection. Then, a feature selection objective function is constructed by combining mutual information and Pearson Correlation Coefficient to select the feature subset with high correlation to the category label and low feature redundancy.
2. A lightweight intrusion detection model: A detection model was developed based on a temporal convolutional network (TCN), which effectively captures the long-term temporal dependency features of network traffic by leveraging its hierarchical causal convolution and dilated convolution structure. Meanwhile, residual connections are utilized to alleviate the vanishing gradient problem, making the model easy

to train and converge. Compared with deep learning models, this method has a smaller parameter scale, faster convergence, and the ability to comprehensively extract sequence features, achieving good detection performance. It can be deployed on resource-constrained edge devices.

3. Validation on the latest dataset: Experimental results on the Edge-IIoTset dataset show that the proposed hybrid feature selection method can significantly reduce feature dimensionality while still maintaining good detection performance. At the same time, compared with other models, TCN has an advantage in classification and detection performance, which verifies the effectiveness of the model.

The rest of this paper is organized as follows. Section 2 introduces the related works about feature selection and IoT intrusion detection method. Section 3 introduces the proposed methodologies, including the feature selection strategies and the TCN model structure design. Section 4 presents the experimental setup and performance evaluation and compares the experimental results with different models. Section 5 discusses the limitations of the model, and Section 6 concludes the paper.

2. Related Works

2.1. Feature Selection Methods for IoT Intrusion Detection

Given that IoT traffic data contains a large amount of redundant and irrelevant features, directly feeding the data into a model for training not only increases the model's computational complexity but also places high demands on the hardware of the deployment environment. To address this issue, many researchers have adopted feature selection methods to achieve data dimensionality reduction.

Traditional feature selection methods are primarily categorized into: filter methods, wrapper methods, and embedded methods [11]. During the feature selection process, the chosen feature subset should have strong classification ability while minimizing redundancy between features. Individual feature selection methods may perform well under specific conditions, but their evaluation methods are too simplistic. Omuya et al. [12] proposed a feature dimensionality reduction method based on principal component analysis. This method requires constructing a covariance matrix of the data and performing eigenvalue decomposition, which has a large computational cost and struggles to meet the real-time processing requirements of large-scale, high-dimensional IoT network traffic data. To reduce computational complexity, some studies have adopted feature selection methods based on statistical properties. Alhassan et al. [13] introduced a novel intrusion detection method based on correlation feature subset selection by calculating the correlation between features and labels. Its detection accuracy is better than that of traditional intrusion detection systems that do not perform feature selection. However, this method only considers the linear correlation between features and categories and fails to consider the nonlinear correlations that exist in network data. Devaraju et al. investigated a feature selection algorithm based on information entropy [14]. This method measures the nonlinear relationship between features and labels by calculating the mutual information value between them. However, in scenarios with small samples or imbalanced class distributions, the stability of mutual information estimation is poor, and the feature selection performance is easily affected.

The Minimum Redundancy Maximum Relevance (mRMR) method, by jointly considering the maximum correlation and minimum redundancy, shows that the inspection performance of the selected feature subset is significantly better than the methods that only use correlation or mutual information [15]. However, this method only evaluates the correlation between features in the subset of candidate features, without considering the correlation between features and categories. Furthermore, since this method requires

calculating the mutual information between features and category labels, it incurs high computational complexity when the sample size is large.

2.2. IoT Intrusion Detection Method Based on Deep Learning

In recent years, with the development of artificial intelligence, NIDS based on deep learning has gradually become a research hotspot. Deep learning constructs multilayer neural networks to realize layer-by-layer feature representation. This approach can automatically extract key features from the original network traffic and capture complex temporal and spatial patterns, thereby achieving effective differentiation between normal and abnormal traffic [16,17].

Ullah et al. [18], introduced an RNN-based method for intrusion detection in IoT networks. A multilayer RNN architecture was designed to exploit the capability of RNNs in modeling temporal sequences, enabling effective capture of time-dependent patterns and anomalous behaviors in network traffic. Reference [19] investigated six deep learning-based models for network attack detection, including multilayer perceptron (MLP), one-dimensional convolutional neural network (1D-CNN), LSTM, gated recurrent unit (GRU), RNN, and a hybrid CNN-GRU model. The six models are systematically evaluated to compare their performance in classifying network traffic into normal or attack. In [20], an intrusion detection scheme was proposed for IoT devices. The authors encoded sensor data through convolution operations to capture patterns within time series, integrating two classical CNN architectures, ResNet and EfficientNet, and evaluated their detection performance. Although these models can capture time-series dependencies in network traffic, they generally suffer from problems such as high detection latency, high computational complexity, and insufficient parallel processing capabilities.

To overcome the limitations of traditional recurrent neural networks in training efficiency and long-distance dependency capture, TCN has been proposed as a new sequence modeling architecture. Compared with traditional deep learning models, TCN uses causal convolution and dilated convolution, combined with residual connections, which enables the network to efficiently capture long sequence dependencies, achieve gradient stable training, and support parallel computing. Inspired by the successful application of temporal convolutional networks in other fields, Lopes et al. [21], designed and implemented four network intrusion detection models based on temporal convolution and conducted a systematic study on their classification performance. Ref. [22] proposed a network intrusion detection method based on TCN. Their experimental results showed that TCN could effectively capture the temporal dependencies of complex network traffic and exhibited efficient and accurate detection performance. Ref. [23] innovatively combined causal and non-causal TCN to construct a hybrid intrusion detection system for the real-time and global prediction needs of IoT networks. The results showed that hybrid TCN improved accuracy by 1.5% and recall by 4% on NSL-KDD, with only a marginal increase of 0.1 ms in detection time. However, most existing studies have focused on limited attack types, and the datasets used do not comprehensively reflect the variety of IoT attacks.

In summary, although existing feature extraction techniques and intrusion detection models have achieved some progress, they still suffer from high computational overhead, suboptimal detection efficiency, and the incomplete coverage of attack types in commonly used datasets. To address these issues, this paper first introduces a hybrid feature selection algorithm to select optimal feature subsets. Subsequently, a lightweight intrusion detection model based on TCN is constructed, and its effectiveness is validated on the latest Edge-IIoTset dataset. This provides a feasible solution for deploying IoT intrusion detection systems on edge servers.

3. Proposed Methodologies

This section presents the overall methodological framework of the proposed approach, as illustrated in Figure 1. As shown in Figure 1, the framework consists of three main stages: data preprocessing, hybrid feature selection, and TCN-based model training. First, the raw dataset is preprocessed through data cleaning and normalization to ensure data quality. Next, a hybrid feature selection module is applied to reduce feature redundancy and retain discriminative features. Finally, the selected feature sequences are fed into the TCN model for training and classification, enabling effective intrusion detection.

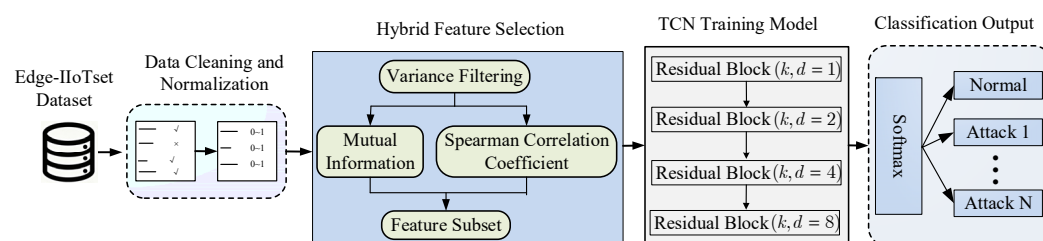


Figure 1. Overall process of the proposed model.

3.1. Dataset Description

In this study, we adopted the Edge-IIoTset dataset, which is widely used for security research in both IoT and Industrial IoT (IIoT) scenarios. Ferrag et al. [24], provide a detailed description of the Edge-IIoTset dataset, which was collected from a realistic IIoT testbed involving more than ten types of IoT devices. For experimental evaluation, we selected the DNN-EdgeIIoT-dataset.csv file from the Edge-IIoTset dataset as the benchmark dataset. This subset provides a large-scale and diverse collection of traffic samples, making it suitable for training and evaluating deep learning-based intrusion detection models. The dataset contains a total of 2,219,201 network flow records, each described by 61 features, covering 14 different attack categories, as well as normal traffic.

3.2. Dataset Preprocessing

To ensure data quality and enhance model training effectiveness, the raw dataset was preprocessed before model construction. As described in Ref. [25], we preprocessed the dataset in the following manner.

3.2.1. Duplicate Row and Unnecessary Feature Removal

First, duplicate rows were removed, resulting in the removal of 815 redundant entries. Second, we removed flow features such as IP addresses, ports, timestamps, and payload data since these primarily serve as network identifiers or payload information, contribute little to attack classification, and may add noise. Specifically, these features are: (1) frame.time, (2) ip.dst_host, (3) http.file_data, (4) ip.src_host, (5) arp.src.proto_ipV4, (6) arp.dst.proto_ipV4, (7) http.request.full_uri, (8) http.request.uri.query, (9) icmp.transmit_timestamp, (10) tcp.payload, (11) tcp.options, (12) tcp.srcport, (13) tcp.dstport, (14) udp.port, and (15) mqtt.msg. After removal of these columns, the dataset contained 1,909,671 records with 47 remaining features.

3.2.2. Dummy Encoding

After dropping duplicate rows and unnecessary features, the new dataset includes seven feature columns with the object data type, apart from the target variable Attack_type. These columns include (1) http.request.method, (2) http.referer, (3) http.request.version, (4) dns.qry.name.len, (5) mqtt.conack.flags, (6) mqtt.protoname, and (7) mqtt.topic. We used dummy encoding to convert these non-numerical features into binary variables. Dummy encoding is a data preparation technique that is widely applied in machine learning and

statistical modeling. This method is very similar to one-hot encoding; the main difference is that one-hot encoding generates k fields for k unique values, while dummy encoding generates $k - 1$ fields for k unique values. That is, in the dummy encoding method, the first unique value line is filled with only 0 values [26]. This helps avoid multicollinearity in regression models while still encoding the categorical information numerically. After dummy encoding, the dataset contained 1,909,671 samples with 97 feature dimensions.

3.2.3. Normalization

Since the original network traffic features exhibited varying value ranges, numerical standardization was performed to ensure that all features shared the same scale. The min–max normalization method was applied, which is described by the following formula:

$$f' = \frac{f - f_{\min}}{f_{\max} - f_{\min}} \quad (1)$$

where f' is the value of f after data normalization.

3.3. Hybrid Feature Selection Algorithm

Figure 2 illustrates the entire feature selection method, which consists of two sequential stages. In the first stage, a variance-based filtering method is applied to the original feature set to remove low-variance features. In the second stage, mutual information and Pearson Correlation Coefficients are further used to screen features. This section will describe in detail the principles of these two feature selection modules.

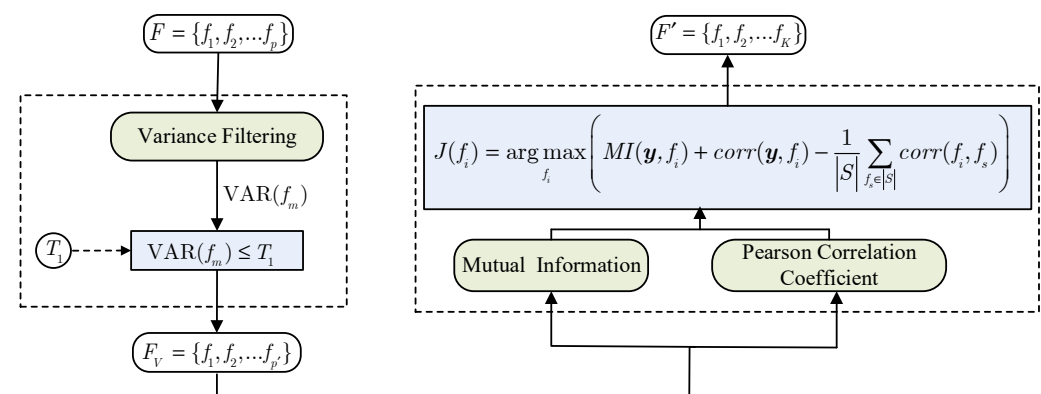


Figure 2. Feature selection process.

3.3.1. Variance-Based Feature Selection

Variance filtering is a statistical method for feature selection. Its core principle is to filter and reduce the dimensionality of features by calculating the variance of each feature in the dataset. When the variance is low, it indicates that the feature value does not vary significantly among samples, the distribution is tightly clustered, and the feature lacks discriminative power. In contrast, high variance suggests the feature has greater discriminative ability. Let f_j be the j -th feature; its variance can be defined as:

$$\text{Var}(f_j) = \frac{1}{N} \sum_{i=1}^N (f_{ij} - \mu_j)^2 \quad (2)$$

where N denotes the number of samples, and μ_j denotes the mean of the j th feature. By using variance filtering to perform preliminary feature selection, initial dimensionality reduction of the network data stream is achieved, which can effectively reduce the complexity of subsequent feature selection.

3.3.2. Mutual Information and Correlation-Based Feature Selection

Variance filtering relies solely on the statistical distribution characteristics of the network data, without considering the relationship between features and labels. To address this issue, mutual information and Pearson Correlation Coefficients are used to measure the relationship between them. Mutual information, a fundamental concept in information theory, is employed to quantify the dependency between two variables. It measures the information that one variable conveys about the other. When knowledge of one variable enables accurate prediction of the other, the mutual information between them is considered high [27]. Assume that $\mathbf{y}$ denotes the corresponding class label in the dataset. According to Ref. [28], the mutual information between the class label and a feature can be expressed as:

$$MI(\mathbf{y}, f_i) = H(\mathbf{y}) + H(f_i) - H(\mathbf{y}, f_i) \quad (3)$$

where $H(\mathbf{y})$ and $H(f_i)$ denote the information entropies of the $\mathbf{y}$ and f_i , respectively, and $H(\mathbf{y}, f_i)$ represents their joint entropy. As shown in Formula (3), a higher mutual information value suggests that a feature preserves more label-related information and thus contributes more effectively to classification performance.

3.3.3. mRMR-Based Feature Selection

The correlation between selected feature subsets can easily introduce feature redundancy, thus affecting the classification performance of intrusion detection models. Influenced by the Minimum Redundancy Maximum Relevance (mRMR) model, Ambusaidi et al. [29] used the Pearson Correlation Coefficient between features and labels to measure the redundancy between features and labels. The expression can be expressed as follows:

$$J_{mRMR}(f_i) = \operatorname{argmax}_{f_i} \left(MI(\mathbf{y}, f_i) - \frac{1}{|S|} \sum_{f_s \in |S|} \frac{\operatorname{corr}(f_i, f_s)}{\operatorname{corr}(\mathbf{y}, f_i)} \right) \quad (4)$$

where $\operatorname{corr}(\mathbf{y}, f_i)$ denotes the Pearson Correlation Coefficient between $\mathbf{y}$ and f_i . It is defined as follows:

$$\operatorname{corr}(\mathbf{y}, f_i) = \frac{\sum_{t=1}^u (\mathbf{y}_t - \bar{\mathbf{y}})(f_{it} - \bar{f}_i)}{\sqrt{\sum_{t=1}^u (\mathbf{y}_t - \bar{\mathbf{y}})^2 (f_{it} - \bar{f}_i)^2}} \quad (5)$$

3.3.4. Hybrid Feature Selection Method

Correlations among features and labels can provide additional information to guide the selection of the features. Consequently, in this article, the Pearson Correlation Coefficient is also employed to evaluate the correlations of features and labels. The overall objective function of the proposed feature selection method is formulated as:

$$J(f_i) = \operatorname{argmax}_{f_i} \left(MI(\mathbf{y}, f_i) + \operatorname{corr}(\mathbf{y}, f_i) - \frac{1}{|S|} \sum_{f_s \in |S|} \frac{\operatorname{corr}(f_i, f_s)}{\operatorname{corr}(\mathbf{y}, f_i)} \right) \quad (6)$$

where S denotes the already selected feature subset, and f_i represents a candidate feature. Based on the above analysis, the hybrid feature selection algorithm proposed in this paper is presented in Algorithm 1.

Algorithm 1 Hybrid Feature Selection Algorithm

Input: Feature set $F = \{f_1, \dots, f_p\}$, class labels $\mathbf{y}$, Number of feature subsets: K

Output: The feature subset F'

Step1. Initialization: set $F' = \emptyset$

Step2. The variance filtering method is used to initially screen the original dataset, resulting in a new dataset $F_V = \{f_1, f_2, \dots, f_{p'}\}$

Step3. Calculate $MI(\mathbf{y}, f_i)$ for each feature in dataset F_V

Step4. Select the feature f_i with the highest mutual information

Step5. Set $F \leftarrow F \setminus f_i$, $F' \leftarrow F' \cup f_i$,

Step6. While $|F'| < K$ do

Calculate $J(f_i)$ in (6) to find f_i , $F \leftarrow F \setminus f_i$, $F' \leftarrow F' \cup f_i$

End For

Step7. Return F'

3.4. Intrusion Detection Framework Based on TCN

The TCN is constructed by stacking multiple residual blocks, with each residual block using a fixed kernel size k and an exponentially increasing dilation factor d . Within each residual block, causal convolution ensures temporal causality, dilated convolution expands the receptive field, and residual connections facilitate model training and convergence. Utilizing the temporal characteristics of network traffic in IoT systems, this paper constructs an intrusion detection model based on TCN. The model diagram is shown in Figure 3.

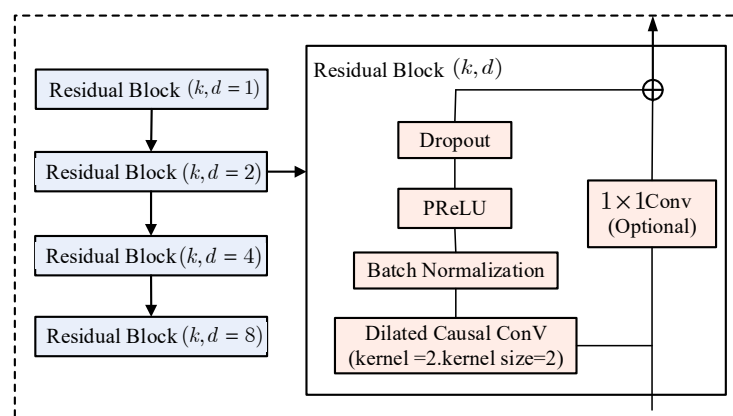


Figure 3. TCN model diagram.

3.4.1. Causal Convolution

Causal convolution restricts the sliding direction of the convolution kernel in the time dimension, ensuring that the output at each time step depends only on the current time step and the previous inputs. Therefore, using causal convolution, the intrusion detection output at time t is only convolved with the time series at time t or earlier, without losing historical data, and can preserve data traffic information over a longer period of time.

3.4.2. Dilated Convolution

Dilated convolutional layers introduce a dilation factor within the convolutional filter, thereby enabling the model to have a larger receptive field. Unlike sequential models, each output position in a dilated convolution depends only on the kernel size and the dilation factor. Therefore, multiple outputs can be computed simultaneously, enabling parallel

computation. This effectively improves the model's prediction efficiency. The definition of dilated convolution is as follows:

$$G(s) = (x \cdot g_d)(s) = \sum_{i=0}^{w-1} g(i) \cdot x_{s-d \cdot i} \quad (7)$$

where g_d denotes the convolution operation with a dilation factor d , w represents the kernel size, and $s - d \cdot i$ denotes the direction toward the past. In TCN, the dilation factor d controls the sampling interval of the convolution kernel along the temporal dimension. As shown in Figure 4, when the kernel size is 2 and the dilation factor $d = 1$, the convolution operation only covers two consecutive time steps, which is a standard one-dimensional convolution. When the dilation factor $d = 2$, the convolution samples input with gaps, expanding the receptive field to three time steps. With $d = 4$, the receptive field is further enlarged, allowing the model to capture information from a longer temporal range. By introducing the dilation factor, TCN can significantly expand the receptive field without increasing the kernel size and the number of model parameters, thereby effectively capturing long-term dependencies in time series.

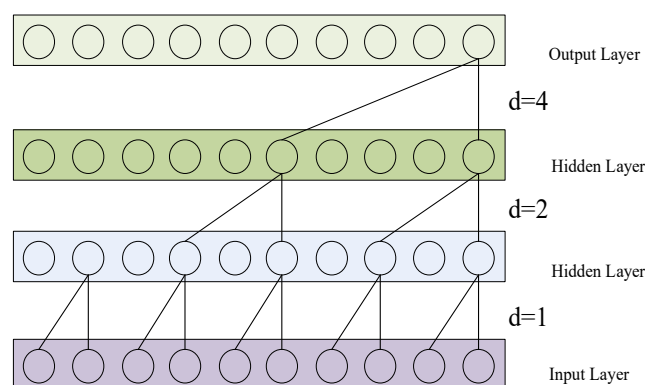


Figure 4. Illustration of dilated causal convolution.

3.4.3. Residual Connection

The residual block simplifies the network training process by introducing residual connections. It typically consists of a series of convolutional layers and shortcut connections, enabling the model to capture long-range dependencies with fewer layers. This design facilitates easier training and faster convergence, mitigates the vanishing gradient problem in deep learning, and enhances both the training stability and generalization performance of the network. The residual block is composed of two branches that operate on the input and output respectively, which can be expressed as follows:

$$o = \delta(x + \varphi(x)) \quad (8)$$

where $\delta(\cdot)$ denotes the activation function, and $\varphi(x)$ represents the transformation composed of four components: dilated causal convolution, weight normalization, activation function, and dropout.

4. Experiments and Performance Evaluation

This section presents the experimental methodology and performance evaluation of the proposed approach. First, the implementation platform, performance evaluation metrics, and hyperparameters used for model training are described. Then, the experimental results of hybrid feature selection and TCN-based classification are discussed in detail.

4.1. Experimental Environment

The experiments were conducted on a workstation equipped with an AMD Ryzen 9 5900X 12-Core Processor CPU and 32 GB RAM. The software environment included Windows 11, Python 3.9.13, and key libraries, such as Tensorflow and Keras version 2.15.0.

4.2. Hyperparameters for the Model's Training

In the training process of deep learning models, hyperparameter settings play a decisive role in the model's convergence speed, performance, and generalization ability. To obtain the best detection results on the dataset used, the experimental parameter settings were set as shown in Table 1.

Table 1. Model hyperparameter configuration.

Parameters	Value
Learning rate	0.001
Optimizer	Adam
Activation function	PReLU
Epochs	50
Batch_size	128
Dilation factor	8
Loss function	Categorical_crossentropy

4.3. Performance Evaluation Metrics

To evaluate the effectiveness of the created model, this paper used accuracy, precision, recall, and F1 score as evaluation metrics for the model training results. The specific calculation formulas are as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (9)$$

$$Precision = \frac{TP}{TP + FP} \quad (10)$$

$$Recall = \frac{TP}{TP + FN} \quad (11)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (12)$$

where TP denotes the number of correctly classified normal instances, TN represents the number of correctly classified attack instances, FP indicates the number of misclassified attack instances, and FN refers to the number of misclassified normal instances.

4.4. Hybrid Feature Selection Algorithm Experiment

In the first step of the hybrid feature selection algorithm, we used the median of the feature variances as the threshold value T_1 . The calculation method was as follows. The variance of each feature in the dataset was calculated, after which these values were sorted in ascending order to identify the median as the threshold T_1 . Features with variances below this value were removed. After this processing, the number of features was reduced from 94 to 47. This step effectively reduced the feature dimensionality and alleviated the computational burden for subsequent feature selection. However, as shown in Figure 5, relatively strong correlations still existed among different features in the retained feature subset.

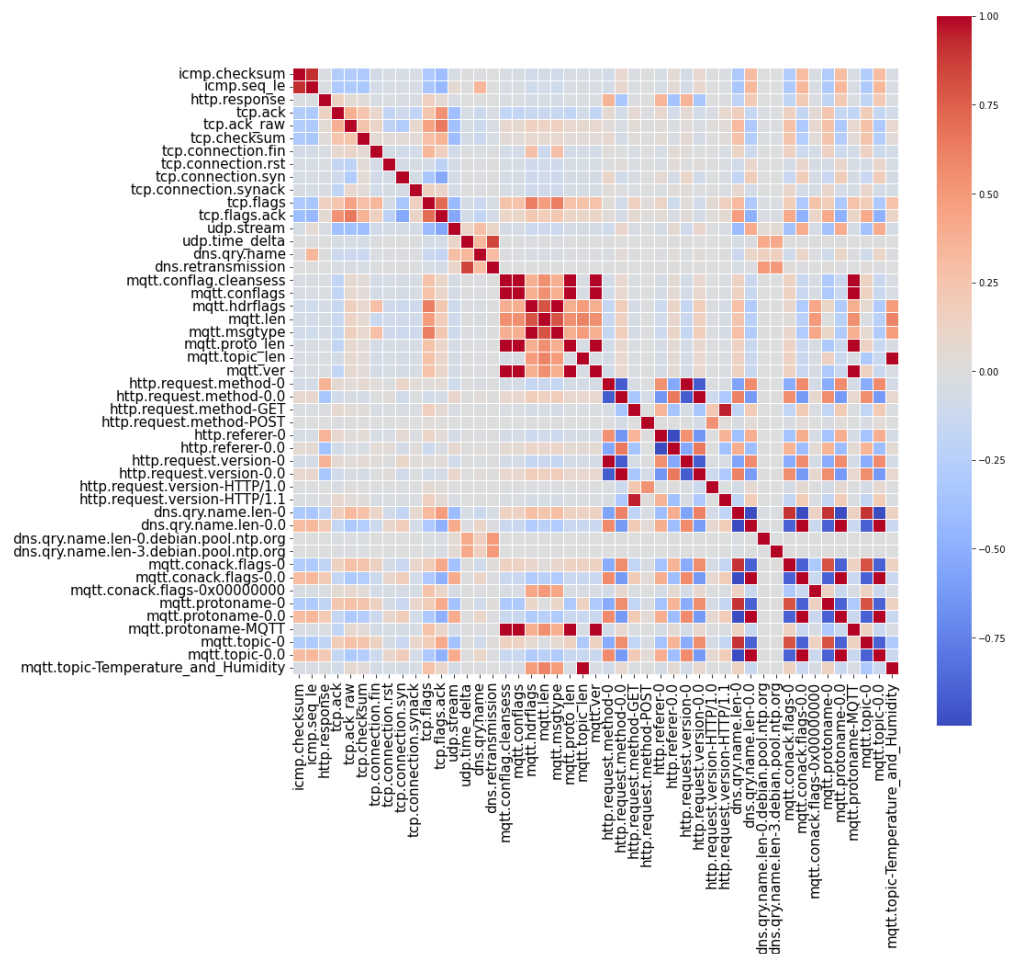


Figure 5. Feature correlation heatmap after variance filtering.

To determine the final number of feature subset K , we selected 6, 12, 16, 18, 20, 22 and all features for the experiment. TCN was used as the classifier. Experimental results in Table 2 show that as the number of features increased, the overall model performance improved, but the computational complexity also increased. Considering both detection performance and computational complexity, we ultimately chose 18 as the feature subset number.

Table 2. Detection results of different feature dimensions (values are in %).

Feature Selection Methods	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Model Parameters (KB)
6	0.79	0.75	0.79	0.76	14.25
12	89.75	90.65	90.65	89.27	15
16	92.35	92.37	92.35	91.52	15.5
18	93.55	92.95	93.55	92.67	15.75
20	93.66	93.16	93.66	92.78	16
22	93.79	93.33	93.79	93.13	16.25
All features	94.24	93.77	94.24	93.71	25.37

After screening using mutual information and Pearson Correlation Coefficients, 18 features were ultimately selected, covering three key categories of traffic attributes. The first category is statistical features, primarily reflecting communication status and packet interaction methods, including icmp checksum.seq_le, tcp.ack, tcp.ack_raw, and udp.stream.

The second category is protocol-native features, extracted from fields in network protocol messages, used to describe protocol behavior, mainly including TCP connection control features, dns query features, and HTTP application layer request and response features. The third category is IoT protocol-related features, primarily selecting features related to the MQTT protocol. These features characterize network traffic behavior from multiple dimensions, thereby enhancing the model's ability to identify abnormal traffic. Appendix A provides a complete description of these features. Figure 6 shows a correlation heatmap of these 18 features. Compared to Figure 5, the correlation between features is significantly reduced, indicating that redundant and highly correlated features were effectively removed.

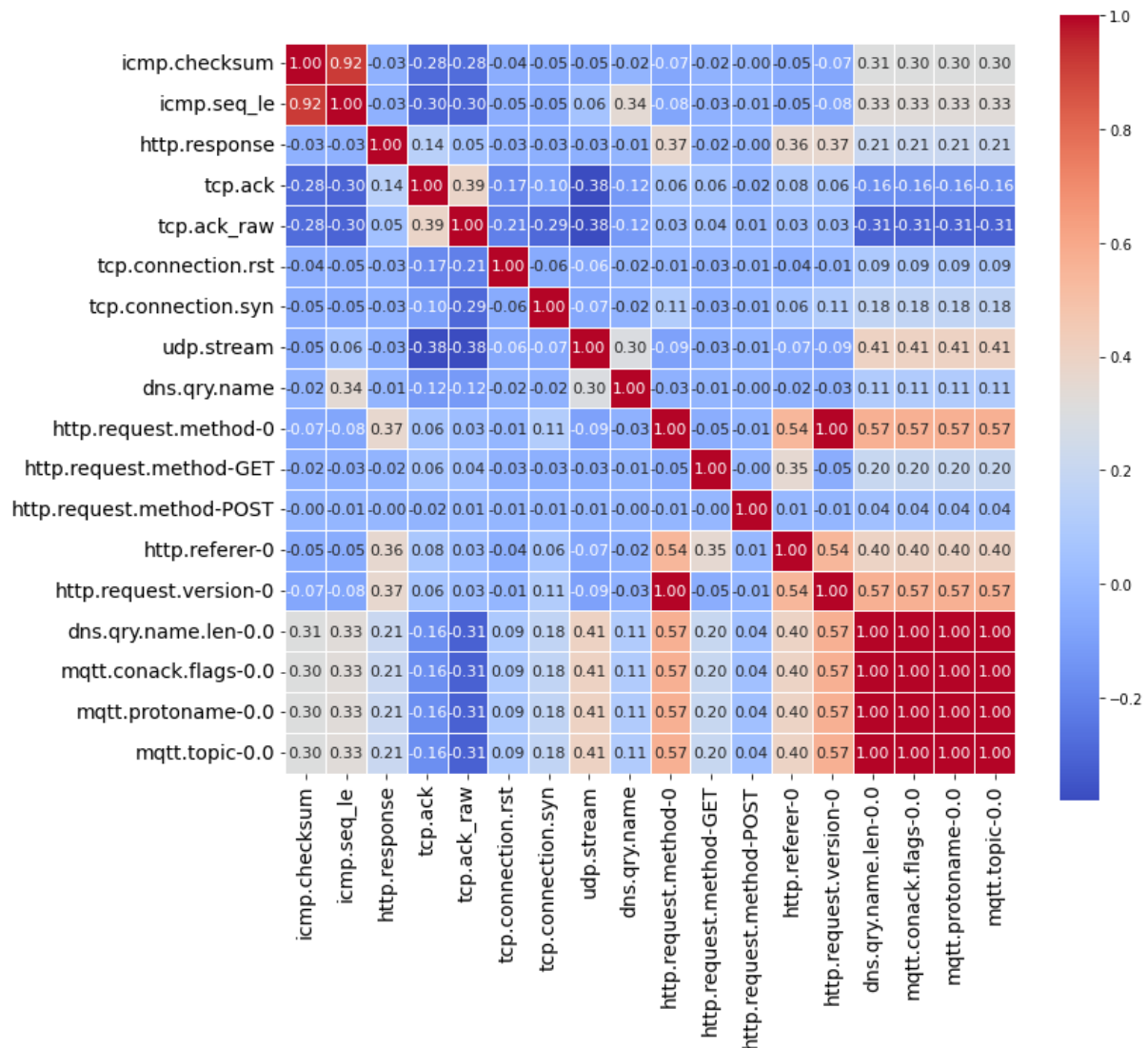


Figure 6. The final selected feature correlation heatmap.

Figure 7 shows the results of the feature selection ablation experiment, where steps 1, 2, 3, and 4 correspond to the mutual information feature selection algorithm, the mRMR method, the proposed method, and all features, respectively. Except for step 4, the number of feature subsets in each step is 18. Experimental results show that the proposed hybrid feature selection algorithm outperforms the mutual information method and the mRMR method.

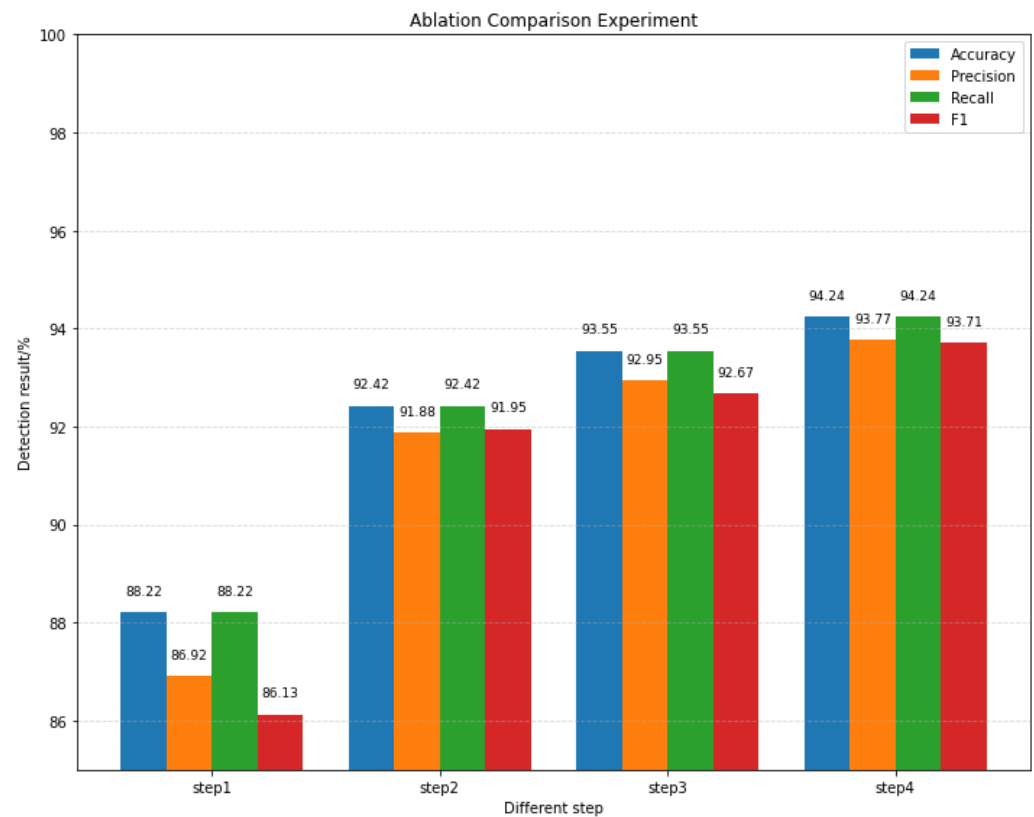


Figure 7. Comparison of ablation experiments.

4.5. TCN Training Experiment Results

Table 3 shows that the TCN training model achieved the best accuracy, recall, and F1 score when the expansion factor d was 8 and the kernel size w was 1. As the expansion factor d increased, the number of layers and parameters in the model increased, leading to an increase in training parameters. Considering all factors, this paper selected a parameter configuration of expansion factor $d = 8$ and kernel size $w = 1$.

Table 3. Test results for different setting of parameters d and w .

d	w	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Model Parameters (KB)
2	1	92.18	92.74	93.18	91.71	9
2	2	93.20	92.20	93.20	92.21	13
2	4	93.19	92.13	93.19	92.29	21
4	1	93.14	93.02	93.14	91.69	12.37
4	2	93.14	92.62	93.14	92.16	18.37
4	4	93.12	92.60	93.12	91.69	30.37
8	1	93.55	92.95	93.55	92.67	15.75
8	2	93.24	92.51	93.24	92.43	23.75
8	4	93.16	92.08	93.16	92.27	39.75
16	1	93.25	92.56	93.25	92.21	19.12
16	2	93.28	92.52	93.28	92.48	29.12
16	4	93.21	92.42	93.21	92.35	49.12

To verify the detection performance of the proposed TCN model, it was compared with various deep learning models, including classic baseline models such as RNN, LSTM, and CNN-LSTM, as well as the latest intrusion detection models mentioned in references [30–33]. Meanwhile, considering the hardware resource limitations of edge servers and the real-time requirements of IoT intrusion detection, the classification performance metrics of each model were compared, including accuracy (Acc.), precision (Pr.), recall (Rc.), and F1 score, as well as the number of model parameters, memory usage, and inference time per sample. All models used the 18 features selected in this paper, and the detection results of the models are shown in Table 4.

Table 4. Comparison of performance of different methods.

Methods	Acc. (%)	Pr (%)	Rc (%)	F1 (%)	Model Parameters (KB)	Memory Usage (MB)	Inference Time (s/inst)
RNN	92.80	91.31	92.80	91.80	24.56	72.77	0.001216
LSTM	91.40	89.39	91.40	89.68	27.43	73.37	0.001390
CNN-GRU [30]	93.04	92.28	93.04	92.20	22.37	40.30	0.001699
CNN-LSTM [31]	92.71	91.13	92.71	91.57	29	103.18	0.001413
CNN-LSTM-ResNet-SA [32]	92.68	91.93	92.68	91.57	25.06	107.87	0.001568
BiGRU-MHA-LSTM [33]	93.09	92.19	93.09	92.86	39.93	78.22	0.005740
TCN	93.79	93.33	93.79	93.13	16.25	39.48	0.000852

In terms of classification performance metrics, the proposed TCN model achieved the highest metrics: 93.79% accuracy, 93.33% precision, 93.79% recall, and a 93.13% F1 score, surpassing the other detection models. These results demonstrate that the TCN effectively captures the long-range temporal dependencies of network traffic through dilated causal convolution, exhibiting stronger classification accuracy in multiple intrusion detection tasks. In terms of resource consumption and real-time performance, the TCN exhibits significant advantages. The model size is only 16.25 KB, which is substantially smaller than that of BiGRU-MHA-LSTM's 39.93 KB. Its memory usage is also relatively low at 39.48 MB, which is significantly lower than CNN-LSTM-ResNet-SA's 107.87 MB and CNN-LSTM's 103.18 MB. Regarding inference efficiency, the inference time per instance is only 0.000852 s (approximately 0.85 ms), making it the fastest among all models. This characteristic is particularly important for resource-constrained IoT intrusion detection scenarios. In summary, the proposed TCN model achieves superior classification performance while maintaining extremely low model size, memory consumption, and inference latency, demonstrating strong potential for deployment in edge servers.

Figure 8 shows the accuracy of each model on the validation set as a function of epochs. The TCN model converges rapidly within 3 to 5 epochs. In contrast, the CNN-GRU and CNN-LSTM-ResNet-SA models also show a relatively rapid upward trend in the early stages, but their final detection accuracy is slightly lower than that of the TCN model. Traditional RNN and LSTM models have lower accuracy in the early stages of training, especially the LSTM model, whose convergence speed is significantly slower than that of other models. As shown in Figure 9, the TCN model exhibits a low validation loss value in the early stages of training, which decreases rapidly in the first few epochs and then stabilizes. The loss of the CNN-GRU and CNN-LSTM-ResNet-SA models decreases faster, but their final loss values are still slightly higher than that of TCN. The loss curve of the GRU_BiLSTM_Attention model is relatively stable, but the overall value is higher than that of TCN. The loss of traditional RNN and LSTM models decreases significantly more

slowly, especially the LSTM model, whose loss value remains at a high level throughout the training process. The combined validation accuracy and loss results show that the TCN model outperforms other comparative models in terms of convergence speed, final classification performance, and training stability. This verifies the advantages of TCN in network intrusion detection tasks.

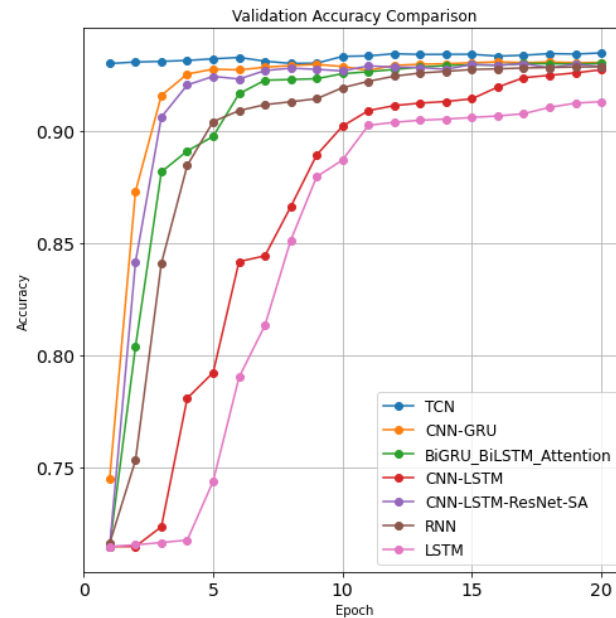


Figure 8. Comparison of validation accuracy across different models.

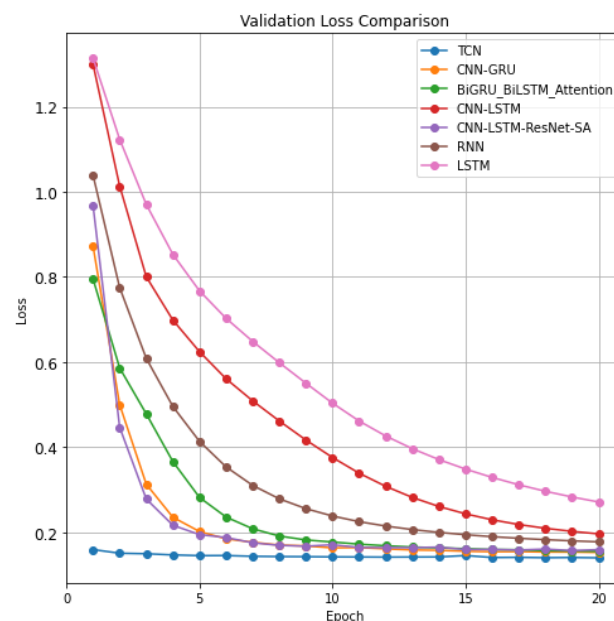


Figure 9. Comparison of validation loss across different models.

5. Discussion

Although the proposed TCN model shows significant advantages over other comparative models in terms of classification performance, convergence, and stability, certain limitations remain when examining the specific detection results for each attack type. As shown in Table 5, the model's accuracy in identifying Normal, DDoS_UDP, DDoS_ICMP, Backdoor, and MITM is close to or reaches 1, indicating that the model effectively learns the characteristics of these traffic types. However, for some attack types, such as Port_Scanning,

SQL_injection, and XSS, the model's detection performance drops significantly. From the confusion matrix shown in Figure 10, Port_Scanning is almost entirely misclassified as DDoS_TCP, and XSS is mostly misclassified as DDoS_HTTP. Table 5 also shows that the number of samples for the aforementioned attack types with poor detection performance is relatively small, exhibiting a significant sample imbalance problem. It is noteworthy that although the MITM attack also has a limited sample size, its detection rate is relatively high. This is because among the 18 selected features, three features mqtt.conack.flags-0.0, mqtt.protoname-0.0, and mqtt.topic-0.0 are highly correlated with the MITM attack. These results indicate that the proposed method has limitations in detecting minority class samples.

Table 5. Detailed classification report.

	Precision	Recall	F1 Score	Support
Backdoor	0.95	0.98	0.96	7193
DDoS_HTTP	0.71	0.98	0.82	14,540
DDoS_ICMP	0.99	0.99	0.99	20,217
DDoS_TCP	0.71	0.99	0.83	15,035
DDoS_UDP	0.99	1.00	0.99	36,523
Fingerprinting	0.80	0.39	0.52	250
MITM	1.00	1.00	1.00	101
Normal	1.00	1.00	1.00	409,327
Password	0.41	0.77	0.55	15,050
Port_Scanning	0.00	0.00	0.00	5987
Ransomware	0.99	0.87	0.93	2892
SQL_injection	0.46	0.20	0.28	15,141
Uploading	0.59	0.34	0.43	11,033
Vulnerability_scanner	0.93	0.84	0.88	15,040
XSS	0.86	0.01	0.02	4535

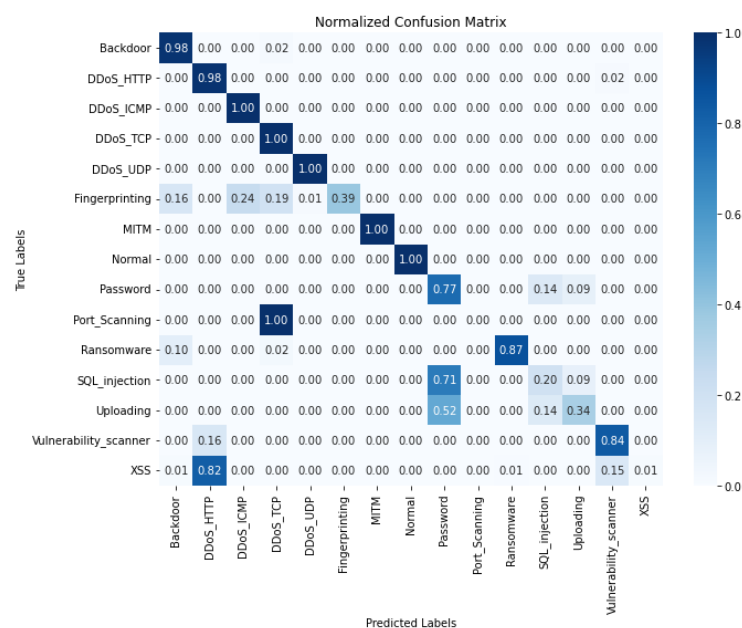


Figure 10. Confusion matrix of classification on the Edge-IIoTset dataset.

Considering these findings, future work will focus on investigating the impact of few samples on feature extraction and the detection performance of the TCN model. Adaptive feature selection strategies or attention-weighted mechanisms can be explored to enhance the contribution of minority samples during training. In addition, edge deployment schemes can be further optimized and combined with federated learning to reduce feature selection and model training costs. To validate the practical feasibility of such optimization strategies, the model's resource consumption and real-time performance on real IoT hardware deserve further evaluation.

6. Conclusions

This paper addresses the challenges of high dimensionality and severe feature redundancy in network traffic data within the IoT environment, as well as the high computational complexity and difficulty in deploying existing intrusion detection models in edge devices. A hybrid feature selection method combined with a lightweight deep learning intrusion detection model is proposed. In the hybrid feature selection stage, variance filtering, mutual information, and Pearson Correlation Coefficient are integrated to effectively reduce feature dimensionality and redundancy, providing a subset of features with strong discriminative power and low feature dimensionality for subsequent model training. Experimental results show that even when the feature dimension is compressed from 94 to 18, the proposed feature selection method still exhibits good detection performance. Based on this, a lightweight intrusion detection model based on TCN is constructed and validated on the Edge-IIoTset dataset. Experimental results demonstrate that the TCN model has significant advantages in detection performance, providing a new approach for deploying intrusion detection system at the edge. In future work, we will investigate the impact of data imbalance to feature selection and TCN model, thereby further enhancing the applicability and robustness of the model for anomalous IoT traffic prediction.

Author Contributions: Conceptualization, W.G. and M.W.; methodology, W.G. and F.L.; software, W.G. and Y.P.; validation, Y.P., C.W. and W.G.; formal analysis, W.G.; investigation, M.W.; resources, C.W.; data curation, C.W.; writing—original draft preparation, W.G.; writing—review and editing, C.W. and M.W.; visualization, Y.P.; supervision, F.L.; project administration, F.L.; funding acquisition, M.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Major Project of Science and Technology Research Program of Chongqing Municipal Education Commission of China (Grant number KJZD-K202302401), the Youth Project of Science and Technology Research Program of Chongqing Education Commission of China (Grant number KJQN202302402), and the Applied Research Project of Chongqing College of Mobile Communication (Grant number KY20240007).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: This work was supported by the Chongqing Key Laboratory of Public Big Data Security Technology.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

IoT	Internet of Things
TCN	Temporal Convolutional Network
NIDS	Network Intrusion Detection System
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory Network
MLP	Multilayer Perceptron
GRU	Gated Recurrent Unit
mRMR	Minimum Redundancy Maximum Relevance
IIoT	Industrial IoT
HTTP	HyperText Transfer Protocol
MQTT	Message Queuing Telemetry Transport
TCP	Transmission Control Protocol

Appendix A

Table A1. Features used for classification.

S. No.	Feature Name	Protocol Layer	Datatype	Description
1	icmp.checksum	ICMP	Unsigned integer	Verifies the integrity of an ICMP message
2	icmp.seq_le	ICMP	Unsigned integer	Helps in identifying the order of sent packets
3	http.response	HTTP	Boolean	Indicates the status code and other information about the response to an HTTP request message
4	tcp.ack	TCP	Unsigned integer	Acknowledges receipt of a TCP segment or sequence of segments
5	tcp.ack_raw	TCP	Unsigned integer	The acknowledgment number in a TCP segment, used for acknowledging receipt of data
6	tcp.connection.rst	TCP	Label	A TCP flag used to indicate that a connection should be reset
7	tcp.connection.syn	TCP	Label	A TCP flag used to initiate a connection
8	udp.stream	UDP	Unsigned integer	A sequence of UDP packets between two hosts
9	dns.qry.name	DNS	Character string	The domain name queried in a DNS request
10	http.request.method-0	HTTP	Character string	Specifies the HTTP method used in HTTP request message
11	http.request.method-GET	HTTP	Character string	Specifies the HTTP method used in HTTP request message
12	http.request.method-POST	HTTP	Character string	Specifies the HTTP method used in HTTP request message
13	http.referer-0	HTTP	Character string	Indicates the URL of the webpage that led to the current request
14	http.request.version-0	HTTP	Character string	Specifies the version of HTTP used in an HTTP request message
15	dns.qry.name.len-0.0	DNS	Unsigned integer	The length of the domain name queried in a DNS request
16	mqtt.conack.flags-0.0	MQTT	Unsigned integer	Flags used in an MQTT connection acknowledgement message
17	mqtt.protoname-0.0	MQTT	Unsigned integer	The name of the MQTT protocol used in an MQTT message
18	mqtt.topic-0.0	MQTT	Character string	The topic published or subscribed to in an MQTT message

References

- Alotaibi, B. A survey on industrial Internet of Things security: Requirements, attacks, AI-based solutions, and edge computing opportunities. *Sensors* **2023**, *23*, 7470. [\[CrossRef\]](#)
- Kumar, M.; Kumar, A.; Verma, S.; Bhattacharya, P.; Ghimire, D.; Kim, S.-H.; Hosen, A.S.M.S. Healthcare Internet of Things (H-IoT): Current trends, future prospects, applications, challenges, and security issues. *Electronics* **2023**, *12*, 2050. [\[CrossRef\]](#)
- Omrany, H.; Al-Obaidi, K.M.; Hossain, M.; Alduais, N.A.M.; Al-Duais, H.S.; Ghaffarianhoseini, A. IoT-enabled smart cities: A hybrid systematic analysis of key research areas, challenges, and recommendations for future direction. *Discov. Cities* **2024**, *1*, 2. [\[CrossRef\]](#)
- Popoola, O.; Rodrigues, M.; Marchang, J.; Shenfield, A.; Ikpehai, A.; Popoola, J. A critical literature review of security and privacy in smart home healthcare schemes adopting IoT & blockchain: Problems, challenges and solutions. *Blockchain Res. Appl.* **2024**, *5*, 100178.
- Kumar, S.; Kumar, D.; Dangi, R.; Choudhary, G.; Dragoni, N.; You, I. A review of lightweight security and privacy for resource-constrained IoT devices. *Comput. Mater. Contin.* **2024**, *78*, 31–63. [\[CrossRef\]](#)
- Chen, K.; Zhang, S.; Li, Z.; Zhang, Y.; Deng, Q.; Ray, S.; Jin, Y. Internet-of-things security and vulnerabilities: Taxonomy, challenges, and practice. *J. Hardw. Syst. Secur.* **2018**, *2*, 97–110. [\[CrossRef\]](#)
- Khraisat, A.; Gondal, I.; Vamplew, P.; Kamruzzaman, J. Survey of intrusion detection systems: Techniques, datasets and challenges. *Cybersecurity* **2019**, *2*, 20. [\[CrossRef\]](#)
- Ahmad, M.; Riaz, Q.; Zeeshan, M.; Tahir, H.; Haider, S.A.; Khan, M.S. Intrusion detection in internet of things using supervised machine learning based on application and transport layer features using UNSW-NB15 data-set. *EURASIP J. Wirel. Commun. Netw.* **2021**, *2021*, 10. [\[CrossRef\]](#)
- Chinnasamy, R.; Subramanian, M.; Easwaramoorthy, S.V.; Cho, J. Deep learning-driven methods for network-based intrusion detection systems: A systematic review. *ICT Express* **2025**, *11*, 181–215. [\[CrossRef\]](#)
- Gyamfi, E.; Jurcut, A. Intrusion detection in internet of things systems: A review on design approaches leveraging multi-access edge computing, machine learning, and datasets. *Sensors* **2022**, *22*, 3744. [\[CrossRef\]](#)
- Pudjihartono, N.; Fadason, T.; Kempa-Liehr, A.W.; O’Sullivan, J.M. A review of feature selection methods for machine learning-based disease risk prediction. *Front. Bioinform.* **2022**, *2*, 927312. [\[CrossRef\]](#)
- Omuya, E.O.; Okeyo, G.O.; Kimwele, M.W. Feature selection for classification using principal component analysis and information gain. *Expert Syst. Appl.* **2021**, *174*, 114765. [\[CrossRef\]](#)
- Alhassan, S.; Abdul-Salaam, G.; Micheal, A.; Missah, Y.M.; Ganaa, E.D.; Shirazu, A.S. CFS-AE: Correlation-based Feature Selection and Autoencoder for Improved Intrusion Detection System Performance. *J. Internet Serv. Inf. Secur.* **2024**, *14*, 104–120. [\[CrossRef\]](#)
- Devaraju, S.; Ramakrishnan, S.; Jawahar, S.; Soni, D.; Somasundaram, A. Entropy-based feature selection for network intrusion detection systems. In *Methods, Implementation, and Application of Cyber Security Intelligence and Analytics*; IGI Global Scientific Publishing: Hershey, PA, USA, 2022; pp. 201–225.
- Xie, S.; Zhang, Y.; Lv, D.; Chen, X.; Lu, J.; Liu, J. A new improved maximal relevance and minimal redundancy method based on feature subset. *J. Supercomput.* **2022**, *79*, 3157. [\[CrossRef\]](#) [\[PubMed\]](#)
- Rahman, M.M.; Al Shakil, S.; Mustakim, M.R. A survey on intrusion detection system in IoT networks. *Cyber Secur. Appl.* **2025**, *3*, 100082. [\[CrossRef\]](#)
- Huang, H.; Wang, P.; Pei, J.; Wang, J.; Alexanian, S.; Niyato, D. Deep learning advancements in anomaly detection: A comprehensive survey. *IEEE Internet Things J.* **2025**, *12*, 44318–44342. [\[CrossRef\]](#)
- Ullah, I.; Mahmoud, Q.H. Design and development of RNN anomaly detection model for IoT networks. *IEEE Access* **2022**, *10*, 62722–62750. [\[CrossRef\]](#)
- Elshewey, A.M.; Abbas, S.; Osman, A.M.; Aldakheel, E.A.; Fouad, Y. DDoS classification of network traffic in software defined networking SDN using a hybrid convolutional and gated recurrent neural network. *Sci. Rep.* **2025**, *15*, 29122. [\[CrossRef\]](#) [\[PubMed\]](#)
- Kodyš, M.; Lu, Z.; Fok, K.W.; Thing, V.L. Intrusion detection in internet of things using convolutional neural networks. In *Proceedings of the 2021 18th International Conference on Privacy, Security and Trust (PST)*, Virtual, 13–15 December 2021; IEEE: Washington, DC, USA, 2021; pp. 1–10.
- Lopes, I.O.; Zou, D.; Abdulqadder, I.H.; Akbar, S.; Li, Z.; Ruambo, F.; Pereira, W. Network intrusion detection based on the temporal convolutional model. *Comput. Secur.* **2023**, *135*, 103465. [\[CrossRef\]](#)
- Nazre, R.; Budke, R.; Oak, O.; Sawant, S.; Joshi, A. A temporal convolutional network-based approach for network intrusion detection. In *Proceedings of the 2024 International Conference on Integrated Intelligence and Communication Systems (ICIICS)*, Karnataka, India, 22–23 November 2024; IEEE: Washington, DC, USA, 2024; pp. 1–6.
- He, P.; Zhang, H.; Feng, Y.; Sakurai, K. A design of network attack detection using causal and non-causal temporal convolutional network. In *Proceedings of the International Conference on Science of Cyber Security*, Melbourne, Australia, 11–14 July 2023; Springer Nature: Cham, Switzerland, 2023; pp. 513–523.

24. Ferrag, M.A.; Friha, O.; Hamouda, D.; Maglaras, L.; Janicke, H. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning. *IEEE Access* **2022**, *10*, 40281–40306. [[CrossRef](#)]
25. Latif, S.; Boulila, W.; Koubaa, A.; Zou, Z.; Ahmad, J. Dtl-ids: An optimized intrusion detection framework using deep transfer learning and genetic algorithm. *J. Netw. Comput. Appl.* **2024**, *221*, 103784. [[CrossRef](#)]
26. Bolikulov, F.; Nasimov, R.; Rashidov, A.; Akhmedov, F.; Cho, Y.-I. Effective methods of categorical data encoding for artificial intelligence algorithms. *Mathematics* **2024**, *12*, 2553. [[CrossRef](#)]
27. Peng, H.; Long, F.; Ding, C. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Trans. Pattern Anal. Mach. Intell.* **2005**, *27*, 1226–1238. [[CrossRef](#)]
28. Coelho, F.; Braga, A.P.; Verleysen, M. A mutual information estimator for continuous and discrete variables applied to feature selection and classification problems. *Int. J. Comput. Intell. Syst.* **2016**, *9*, 726–733. [[CrossRef](#)]
29. Ambusaidi, M.A.; He, X.; Nanda, P.; Tan, Z. Building an intrusion detection system using a filter-based feature selection algorithm. *IEEE Trans. Comput.* **2016**, *65*, 2986–2998. [[CrossRef](#)]
30. Saadouni, R.; Khacha, A.; Harbi, Y.; Gherbi, C.; Harous, S.; Aliouat, Z. Secure IIoT networks with hybrid CNN-GRU model using Edge-IIoTset. In *Proceedings of the 2023 15th International Conference on Innovations in Information Technology (IIT)*, Al Ain, United Arab Emirates, 14–15 November 2023; IEEE: Washington, DC, USA, 2023; pp. 150–155.
31. Khacha, A.; Saadouni, R.; Harbi, Y.; Aliouat, Z. Hybrid Deep Learning-based Intrusion Detection System for Industrial Internet of Things. In *Proceedings of the 2022 5th International Symposium on Informatics and its Applications (ISIA)*, M’sila, Algeria, 29–30 November 2022; pp. 1–6.
32. Sasi, T.; Lashkari, A.H.; Lu, R.; Xiong, P.; Iqbal, S. An efficient self attention-based 1D-CNN-LSTM network for IoT attack detection and identification using network traffic. *J. Inf. Intell.* **2024**, *3*, 375–400. [[CrossRef](#)]
33. Gueriani, A.; Kheddar, H.; Mazari, A.C.; Ghanem, M.C. A robust cross-domain IDS using BiGRU-LSTM-attention for medical and industrial IoT security. *ICT Express* **2025**, *in press*.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.